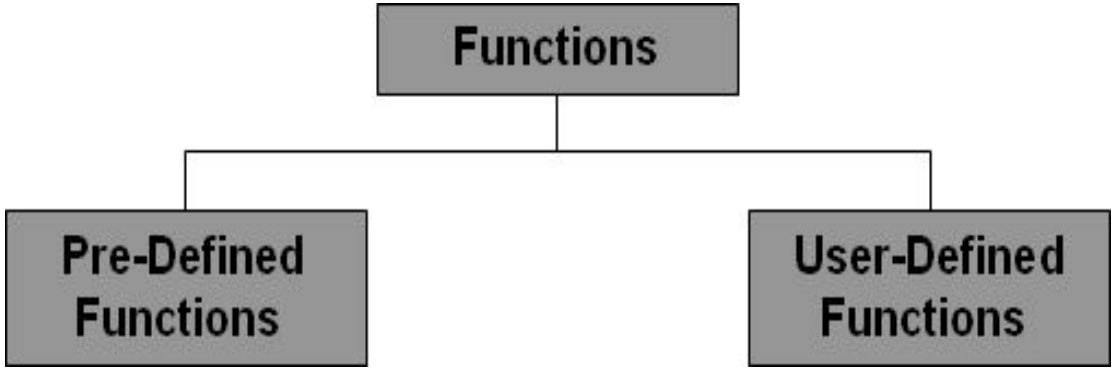
FUNCTIONS

Agenda

- Functions
- Library and user defined functions
- Need for user defined functions
- Merits of functions
- Elements of a function
- Categories of function
- References

What is a Function?

- A function is a set of statements that take inputs, do some specific computation and produces output.
- A large C program is divided into basic building blocks called C function.
- C function contains set of instructions enclosed by "{ }" which performs specific operation in a C program.
- Collection of functions creates a C program.
- C functions can be classified into two categories
  - Library functions (Pre-defined Functions)
  - User-defined functions



LIBRARY & USER DEFINED FUNCTIONS

Library Functions

- Library functions are not required to be written by us.
- printf and scanf belong to the category of library function.
- sqrt, cos, strcat, etc are some of the library functions.

User-defined Functions

- User-defined functions has to be developed by the user at the time of writing a program.
- User-defined functions can later become a part of the C program library.
- main() is an example of user-defined functions.

Library (Built In) Functions:

- They are written in the header files.
- To use them, appropriate header files should be included.

Header Files	Functions Defined
stdio.h	printf(), scanf(), getchar(), putchar(), gets(), puts(), fopen(), fclose()
conio.h	clrscr(), getch()
ctype.h	toupper(), tolower(), isalpha()
math.h	pow(), sqrt(), cos(), log()
stdlib.h	rand(), exit()
string.h	strlen(), strcpy(), strupr()

User Defined Functions

- You can also create functions as per your need.
- Such functions created by the user are known as user-defined functions.

```
#include <stdio.h>
void functionName();
int main()
{    ... .. ...  ... .. ...
  functionName();
 ... .. ... ... .. ...
}
void functionName()
{
  ... .. ..... .. ...
}
```

Without Function

```
int main( )
{
   printf(“----------\n”);
   printf(“Hello\n”);
   printf(“----------\n”);
   printf(“Hai\n”);
   printf(“----------\n”);
   printf(“Greetings of the day\n”);
   printf(“----------\n”);
}
```

OUTPUT

```
----------
Hello
----------
Hai
----------
Greetings of the day
----------
```

With Function

```
int main( )
{
   printline(); // calls the printline procedure
   printf(“Hello\n”);
   printline();
   printf(“Hai\n”);
   printline();
   printf(“Greetings of the day\n”);
   printline();
}
printline()
{printf(“----------\n”);
}
```

OUTPUT

```
----------
Hello
----------
Hai
----------
Greetings of the day
----------
```

NEED FOR USER-DEFINED FUNCTIONS

- Every program must have a main function.
- It is possible to code any program utilizing only main function, but it leads to a number of problems.
- The program may become too large and complex and as a result the task of debugging, testing, and maintaining becomes difficult.
- If a program is divided into functional parts, then each part may be independently coded and later combined into a single unit.
- These subprograms called ‘functions’ are much easier to understand, debug, and test.
- There are times when some types of operation or calculation is repeated at many points throughout a program. Then, we may repeat the program statements whenever they are needed.
- Another approach is to design a function that can be called and used whenever required.
- This saves both time and space.

NEED FOR USER-DEFINED FUNCTIONS

- This sub-sectioning approach clearly results in a number of advantages:
  - It facilitates top-down modular programming.
  - The length of a source program can be reduced by using functions at appropriate places. This factor is particularly critical with microcomputers where memory space is limited
  - It is easy to locate and isolate a faulty function for further investigations.
  - A function may be used by many other programs, this means that a C program can build on what others have already done, instead of starting over, from scratch.

Uses/Merits of Functions

- C functions are used to avoid rewriting same logic/code again and again in a program.
- There is no limit in calling C functions to make use of same functionality wherever required.
- We can call functions any number of times in a program and from any place in a program. Functions can be used by other program's.
- Modular Programming.
- A large C program can easily be tracked when it is divided into functions. Dividing a big task into small pieces to achieve the functionality and to improve understandability of very large C programs.
- Functions can be re-used.
- Length of source program can be reduced.
- Easy to locate and isolate faulty functions.

Syntax

- The general form of a function definition in C programming language is as follows −

```
return_type function_name( parameter list )
 {
   body of the function
}
```

Here are all the parts of a function −

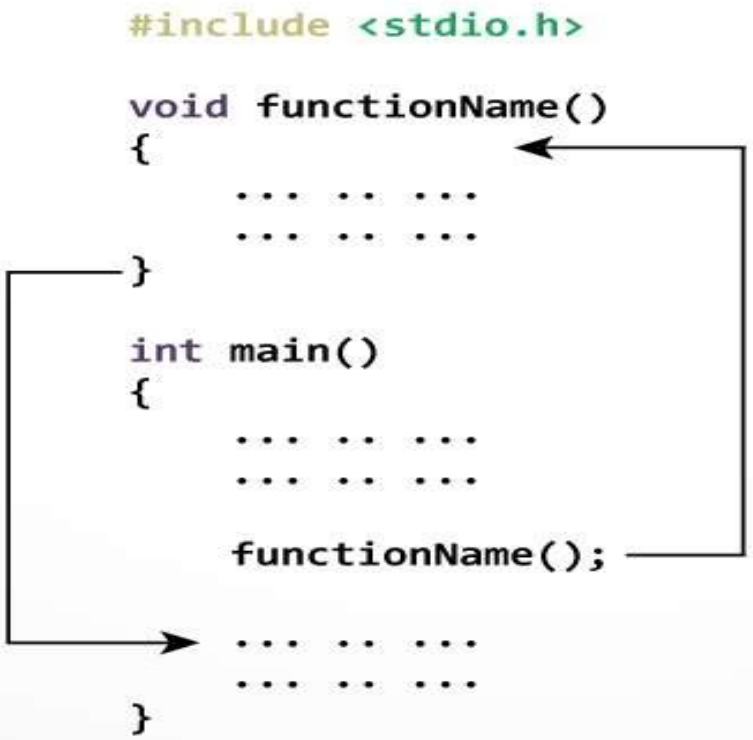
Return Type − A function may return a value. The **return_type** is the data type of the value the function returns. Some functions perform the desired operations without returning a value. In this case, the return_type is the keyword **void**.

Function Name − This is the actual name of the function. The function name and the parameter list together constitute the function signature.

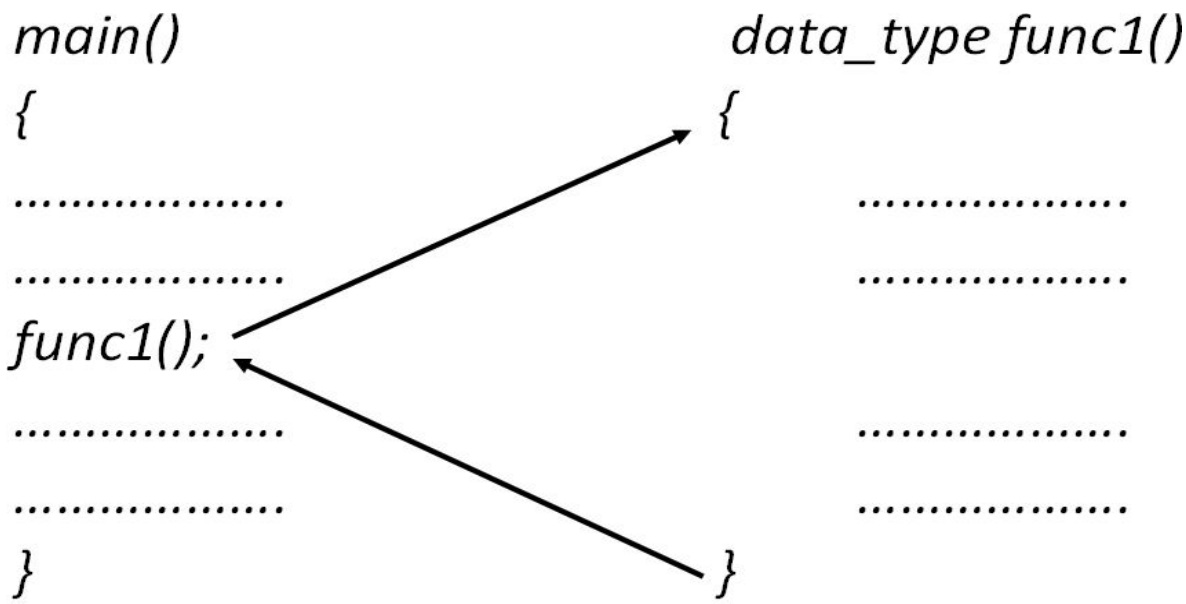
Parameters − A parameter is like a placeholder. When a function is invoked, you pass a value to the parameter. This value is referred to as actual parameter or argument. The parameter list refers to the type, order, and number of the parameters of a function. Parameters are optional; that is, a function may contain no parameters.

Function Body − The function body contains a collection of statements that define what the function does.

How function works in C programming?



FUNCTION



Calling function

Called function

Elements of User defined functions

- Function Prototype (Declaration)
- Function Call
- Function arguments and parameters
- Function Definitions

Function prototype

- A function prototype is simply the declaration of a function that specifies function's name, parameters and return type.
- It doesn't contain function body.
- A function prototype gives information to the compiler that the function may later be used in the program.

Syntax:

- return-type name-of-function(list of arguments);
- Example

```
void sum(int, int);
```

Function Call

- A function can be called by specifying name and list of arguments enclosed in parenthesis and separated by comma.
- If there is no argument, empty parenthesis are placed after function name.
- If function return a value, function call is written as assignment statement as:

```
A=sum(x,y);
```

Function arguments and parameters

Arguments

- Arguments are also called actual parameters.
- Arguments are written within parenthesis at the time of function call.

Parameters

- Parameters are also called formal parameters.
- These are written within parenthesis at the time of function definition.

Function Definition

- It is the independent program module.
- It is written to specify the particular task that is to be performed by the function.
- The first line of the function is called function declarator and rest line inside { } is called function body.

```c
#include<stdio.h>
#include<conio.h>
void sum(int,int); // function prototype
void main()
{
 int a,b;
 printf("Enter the two integers");
 scanf("%d%d",&a,&b);
 sum(a,b); // function call
 getch();
}
void sum(int a, int b) // function defination
{
 printf("The sum of the numbers you entered is %d",a+b);
}
```

Arguments

Parameters

CALLING A FUNCTION

```
int mul (int, int);
main()
{
      int p;
      p = mul(10,5);
      printf("%d\n", p);
}
int mul(int x,int y)
{
     int p;          /*local variables*/
     p = x * y;         /* x = 10, y = 5*/
     return(p);
}
```

Return statement

- It is the last statement of the function that return certain values.
- It return certain types of values to the place from where the function was invoked.
- Syntax:
  - return(variable-name or constant);

Categories of function

1. Functions with no arguments and no return values
2. Functions with arguments and no return values
3. Functions with no arguments but return a value
4. Functions with arguments and one return value
5. Functions that return multiple values

1. Functions with no arguments and no return values

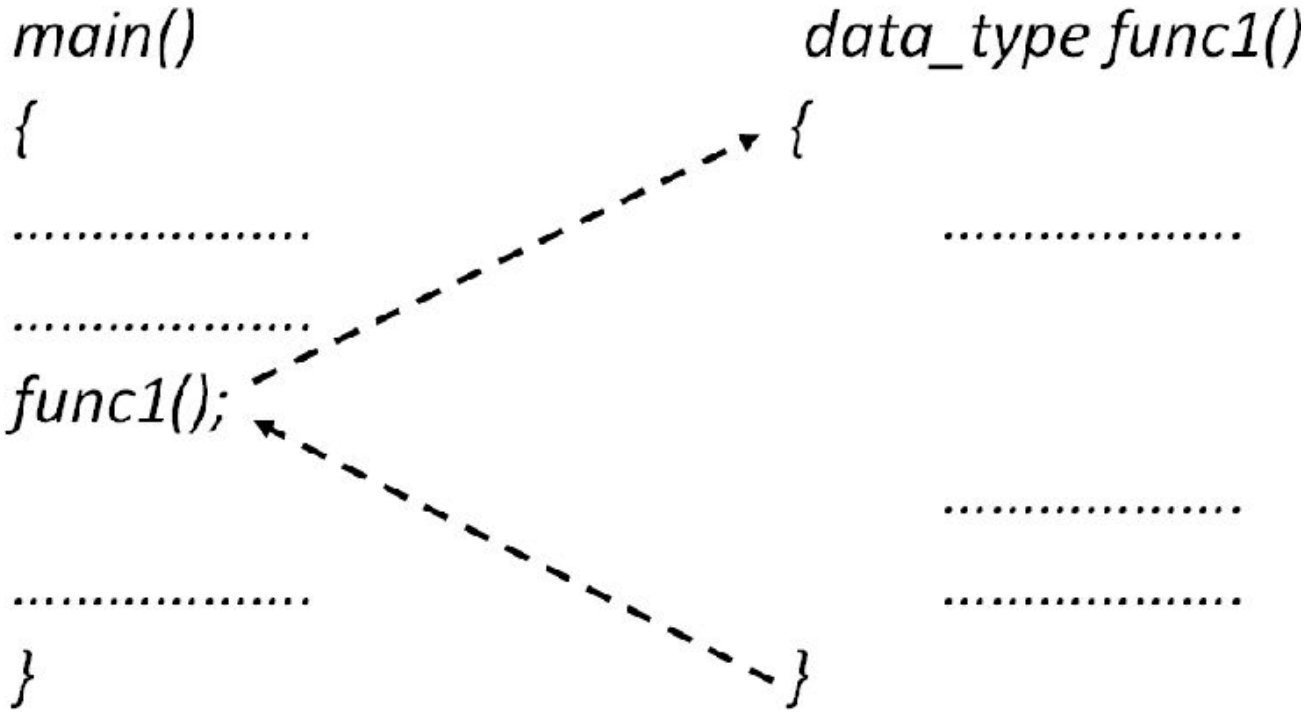
```
#include<stdio.h>
#include<conio.h>
void sum();
void main()
{
 sum();
 getch();
}
void sum()
{
 int a,b;
 printf("Enter the two integers");
 scanf("%d%d",&a,&b);
 printf("The sum of the numbers you entered is %d",a+b);
}
```

OUTPUT:

Enter the two integers: 13 6

The sum of the two numbers you entered is 19

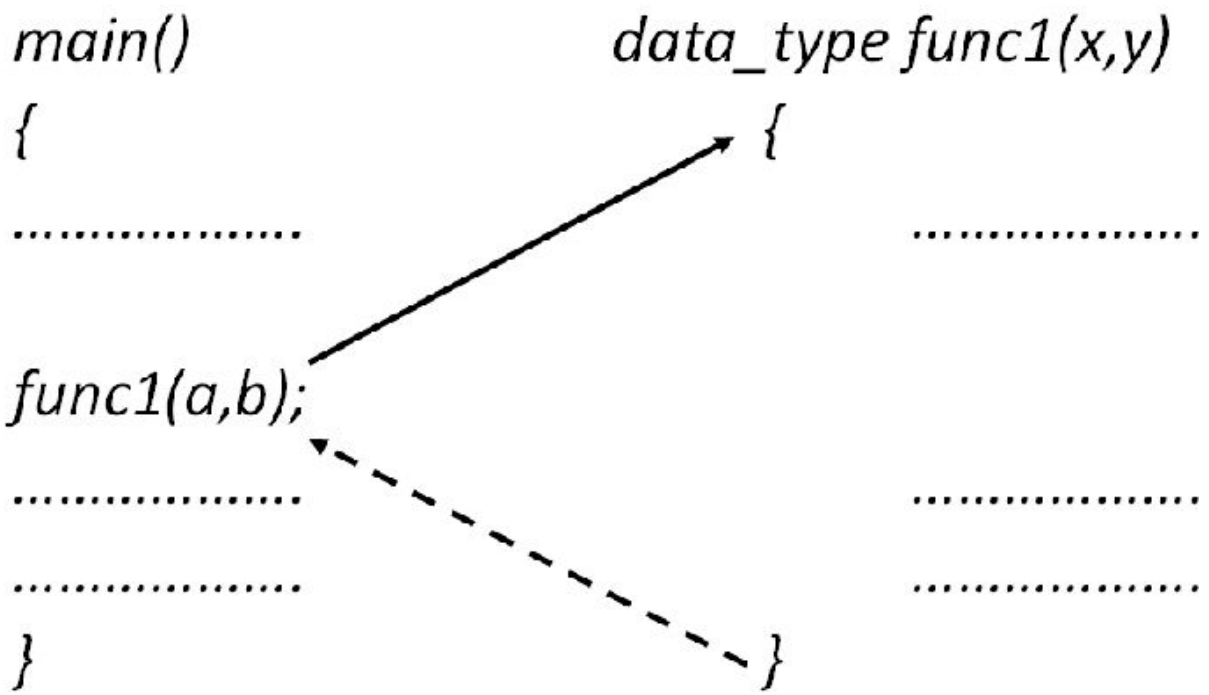
Function with no arguments and no return values

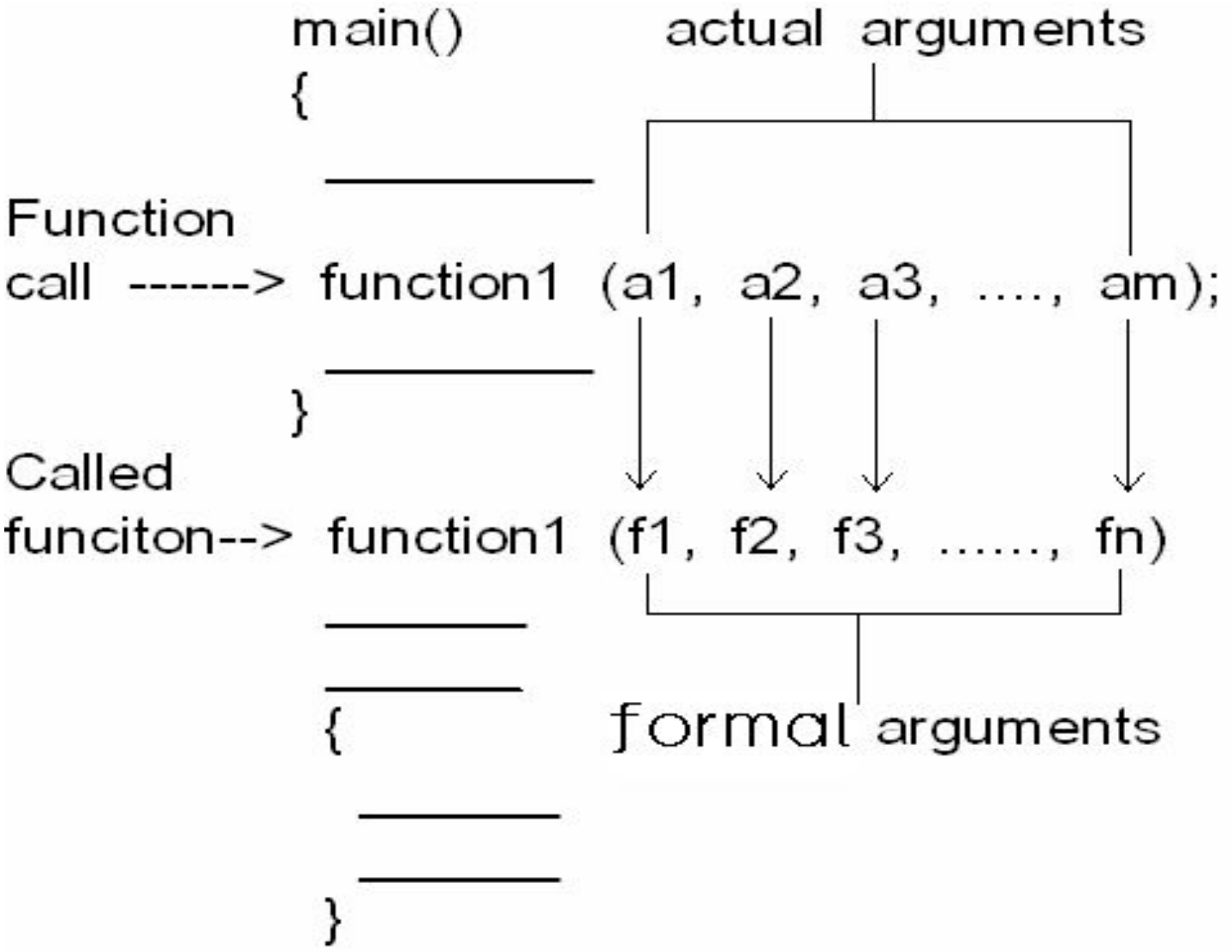


2. Functions with argument and no return values

```
#include<stdio.h>
#include<conio.h>
void sum(int,int);
void main()
{
 int a,b;
 printf("Enter the two integers");
 scanf("%d%d",&a,&b);
 sum(a,b);
 getch();
}
void sum(int a, int b)
{
 printf("The sum of the numbers you entered is %d",a+b);
}
```

Function with arguments and no return values




main()
{
actual arguments
Function
call ------> function1 (a1, a2, a3, ..., am);
}
Called
funciton--> function1 (f1, f2, f3, ..., fn)
{
formal arguments
}

EXAMPLE

```
#include <stdio.h>
#include<conio.h>
mul(int, int);
void main()
{
    int a,b;
    printf("\nEnter two number:");
    scanf("%d%d",&a,&b);
    mul(a,b);
}
```

```
//function with arguments
mul(int x, int y)
{
 int z;
 z=x*y;
 printf("\nProduct  is:%d",z);
 }
```

OUTPUT:

Enter two number:2 4

Product is: 8

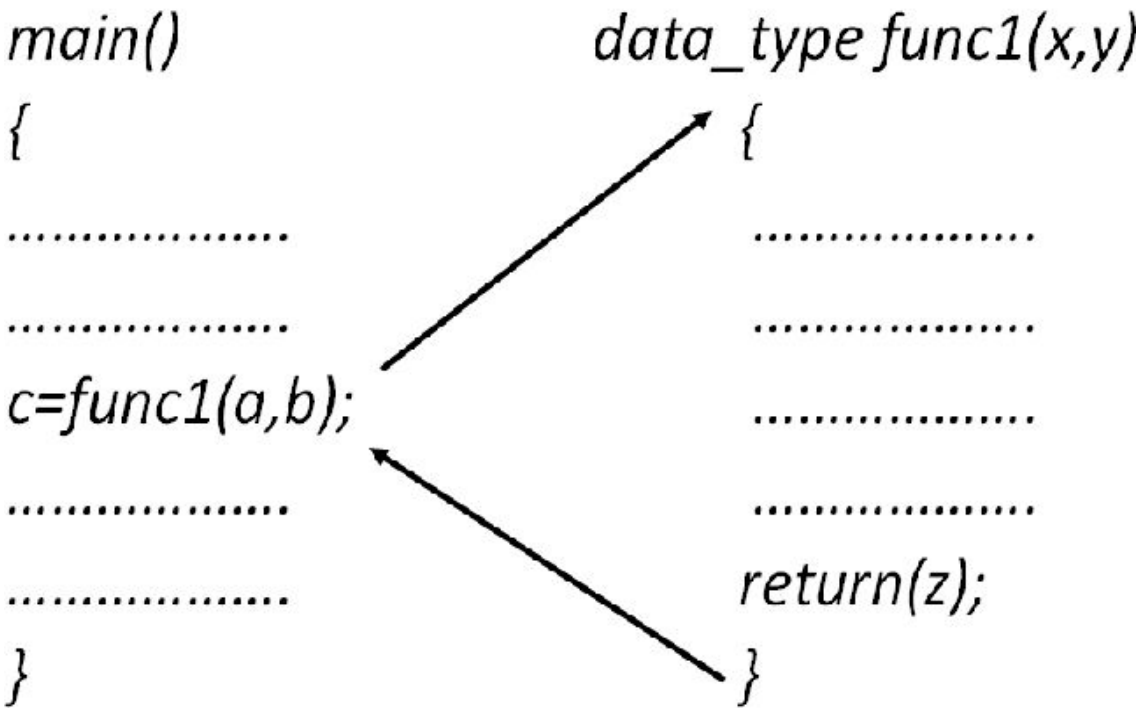
3. Functions with no arguments but return a value

```
#include<stdio.h>
#include<conio.h>
int sum();
void main()
{
 int x;
 x=sum();
 printf("The sum of the numbers you entered is %d",x);
 getch();
}
int sum()
{
 int a,b;
 printf("Enter the two integers");
 scanf("%d%d",&a,&b);
 return(a+b);
}
```

4. Functions with arguments and one return value

```
#include<stdio.h>
#include<conio.h>
int sum(int,int);
void main()
{
 int a,b,x;
 printf("Enter the two integers");
 scanf("%d%d",&a,&b);
 x=sum(a,b);
 printf("The sum of the numbers you entered is %d",x);
 getch();
}
int sum(int a, int b)
{
 return(a+b);
}
```

Function with arguments and return values



EXAMPLE

```
#include <stdio.h>
#include<conio.h>
float avg(int, int);
void main()
{
    int a,b;
    float c;
    printf("\nEnter two numbers:");
    scanf("%d%d",&a,&b);
    c=avg(a,b);
    printf("\nAverage is:%f",c);
}
```

```
float avg(int x, int y)
{
 float z;
 z=x+y/2;
 return(z);
}
```

OUTPUT:

```
Enter two numbers:6  7

Average is:6.5
```

5. Functions that return multiple values

- A return statement can return only one value.
- To get more information from a function, we can achieve this in C using the arguments not only to receive information but also to send back information to the calling function.
- The arguments that are used to "send out" information are called *output parameters*.
- The mechanism of sending back information through arguments is achieved using what are known as the *address operator (&) and indirection operator (*)*.

Functions that return multiple values

```
void mathoperation (int x,int y, int *s, int *d);
main()
{
        int x=20, y=10,s, d;

      mathoperation(x, y, &s, &d);
      printf("s=%d\n  d=%d\n", s, d);
}

void mathopertaion (int a, int b, int *sum, int *diff)
{
      *sum=a+b;
      *diff=a-b;
}
```

OUTPUT

s=30

d=10

- x and y – input arguments
- s and d – output arguments
- We pass
  - Value of x to a
  - Value of y to b
  - Address of s to sum
  - Address of d to diff
- The body of the function when executed add the value and find difference and store the result in memory location pointed by *sum* and *diff.*

Functions that return multiple values

- The indirection operator * indicates the variables are to store addresses, not actual values of variables.
- The operator * is known as indirection operator because it gives an indirect reference to a variable through its address.
- The variables *sum and *diff are known as *pointers* and sum and diff as *pointer variables*.
- They are declared as int, they can point to locations of int type data.
- The use of pointer variables as actual parameters for communicating data between functions is called *"pass by pointers"* or *"call by address or reference"*.

Factorial program in C using function

```
#include <stdio.h>

long factorial(int);

int main()
{
 int number;
 long fact;

printf("Enter a number to calculate its factorial\n");
scanf("%d", &number);

fact=factorial(number);

printf("%d! = %ld\n", number, fact);
return 0;
}
```

```
long factorial(int n)
{
 int c;
 long result = 1;

 for (c = 1; c <= n; c++)
      result = result * c;

 return result;
}
```

OUTPUT

Enter a number to calculate its factorial
4
4!=24

References

- https://www.slideshare.net/ijs2k8/programming-in-c-70043397
- https://www.slideshare.net/gauravjuneja11/c-language-ppt

THANK YOU